



Computer Vision Final Lab Part 2



October 18, 2024

Students:

Alisia Baielli
UvAnetID 15823725

Sander Hoffman
UvAnetID 13995995

Maxim Voronin
UvAnetID 15667669

Group:

Group 26

Course:

Computer Vision 1

1 Introduction

Image classification is a task in which we are given some images and asked to assign a class from a predetermined set of possible classes. It is a very relevant problem for many computer vision applications. In this report we are going to analyse the image classification pipeline using convolutional neural networks (CNNs). CNNs are often used for image classification, because they can learn local features while often using less parameters than fully connected neural networks. Besides that we will also be looking at the effectiveness of knowledge transfer at our scale.

2 Method

2.1 Architecture

To analyse image classification with neural networks we will initially be training two neural networks. One fully connected neural networks with a single hidden layer and the convolutional neural network LeNet-5[3]. For the fully connected neural network we can choose the hidden layer size when it is created. The input should be of size $3 \times 32 \times 32$, which gets flattened to be able to be read by the neural network. The LeNet-5 architecture is visualised in figure 1.

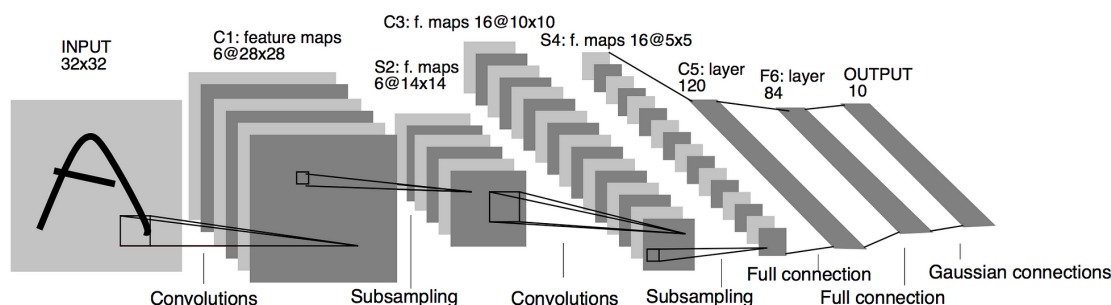


Figure 1: The architecture of LeNet-5 as reported by LeCun et. al.[3]

A slight difference with our implementation is that the input and output is of a different shape. This is because, instead of working with the MNIST dataset like in the original LeNet

paper, we will initially be working with CIFAR-100[2]. This dataset contains 600 images per class, 100 test and 500 training, for a total of 60000 32x32 images in the dataset. A visualisation of the CIFAR-100 dataset is shown in figure 15. The CIFAR-100 dataset contains 50000 32x32 RGB images. This means instead of the input being of shape 1x32x32, our input will be of shape 3x32x32. CIFAR-100 also has 100 classes instead of the 10 classes that MNIST has, so our output layer will be 100 nodes instead of 10. This means that the amount of parameters in the F6 layer changes from 850 to 8500. Another two differences our architecture has when compared to the original LeNet-5, is that LeNet-5 used average pooling and a sigmoid activation function, while we use max pooling and the ReLU activation function. We chose these because they are more widely used in neural networks today.

To increase the expressiveness of the networks we were allowed to add two layers. These layers had to be either fully connected or convolutional layers. For the fully connected network with a single hidden layer, we decided to add a convolutional layer with a kernel size of 5 with max pooling to the start and a fully connected layer after the first hidden layer. The convolutional layer should allow the network to become more translation invariant, while the extra linear layer will allow for better predictions. For the Convolutional network we decided to add an extra convolutional layer after the third and an extra fully connected layer between C5 and F6. This layer has a kernel size of 3 and takes 16 channels, which it turns into 48 channels. This increase in channels allows for more kinds of features to be learned. When this extra convolution is used the kernel size of the previous convolutional layer is reduced to 3x3, so that the output size stays the same. The extra fully connected layer allows the now deeper network to make more complex relations easier.

2.2 Hyperparameter Search

In this section we discuss all hyperparameters that we varied and what we expect them to do. All models were trained for 25 epochs while varying one of the parameters. The others were frozen to their default value which we mention below for each parameter.

- Optimiser:

The optimiser determines which algorithm is used to calculate gradients and alter the model weights. We will be testing the Adaptive Moment Estimation (ADAM) optimiser and the Stochastic Gradient Descent (SGD) optimiser. As ADAM is a more advanced version of SGD, we expect ADAM to perform better overall. The default value is ADAM.

- Learning rate:

The learning rate determines how fast the weights of the model change based on the gradients found by the optimiser. A high learning rate means more change, but can be detrimental to the convergence of the network, while a slow learning rate means we need more epochs to find good weights. The values we test are 0.01, 0.001, and 0.0001. The default value is 0.001.

- Batch size:

The batch size determines how many entries get fed into the network before gradients are calculated. Low batch sizes introduce higher variance into the model, which may hinder convergence, but also allows models to escape local minima more easily. Higher batch sizes make the gradient updates more stable and leads to faster convergence, but might decrease the generality of the final model. This is because the model gets stuck in local minima more easily, which may decrease performance on the test set. For this parameter we test the values 16, 64 and 512. The default value is 64.

- Weight decay:

Weight decay introduces a penalty on higher model weights. The effect of this is that with higher weight decay, overfitting is reduced, but so is the expressiveness of the model. Thus, a good value for weight decay successfully prevents overfitting while still allowing the model to mostly map the data to the correct labels. The values we test for this parameter are 0, 0.01 and 0.1. The default value is 0.

Transformation presets visualisation

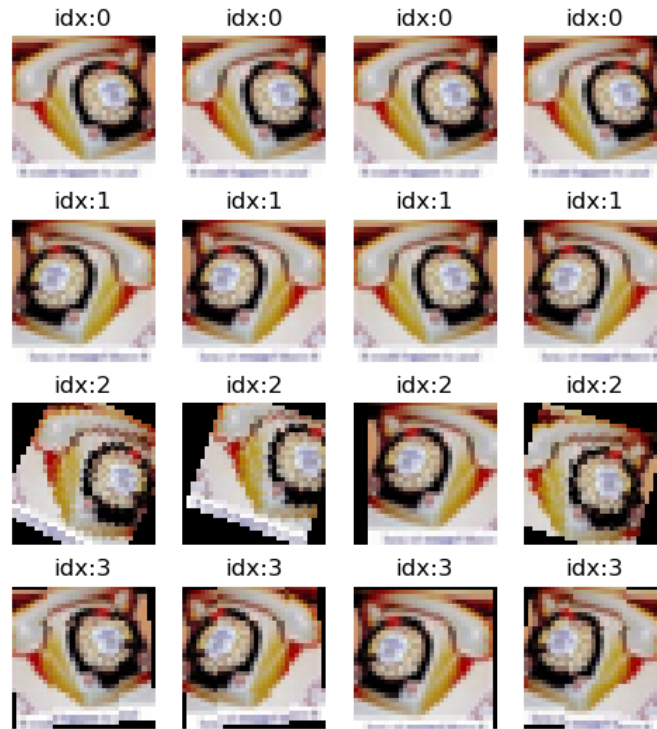


Figure 2: The same image from CIFAR-100 when put through a transformation sampled 4 times.

- **Batch normalisation:** Batch normalisation allows training to be more stable and causes faster convergence. It can also decrease the influence of the random initialisation of the model with regard to the final model. When using batch normalisation, one batch normalisation layer is added between the hidden and the output layer for the fully connected network, and for the convolutional neural network one batch normalisation layer is added after both the first and second convolution. We test both with and without batch normalisation. The default value is without batch normalisation.
- **Transformations:** Transforming images randomly during training can increase the generality of models greatly, as the data between epochs is never exactly the same. This forces the network to learn trends and local features instead of just memorizing parts of the training set. For the transformations on the input data, we created four variants. The first does nothing to the image. The second randomly flips the image horizontally and randomly changes brightness, saturation and hue of a pixel by 1 percent. The third is the same as the second except it also shifts by up to four pixels and rotates up to 30 degrees either clockwise or anti-clockwise. The final is the same as the third except the shift is reduced to 1 pixel and the max rotation to 5 degrees. All transformations are visualised in figure 2. We also conclude each transformation by changing the images to be in the range $[-1,1]$, which could aid the models. The default value is transformation idx 0 (No transformations except range change).

2.3 STL-10

To see how effective transfer learning can be, we take the best model we trained on CIFAR-100 and adapt the input and output to work on a different dataset. Because we trained the middle layers on CIFAR-100, we should be able to get good performance on the new dataset relatively

quickly when compared to training a model from scratch. To test this, we are working with the STL-10[1] dataset. This dataset contains 500 labelled training images per class and 800 test images. We will be training only on the Bird, Deer, Dog, Horse, and Monkey classes, which we visualise in figure 14, thus the total number of images in our dataset becomes 6500, of which 4000 belong to the test set. We will use the convolutional neural network with an increased amount of layers and the best hyperparameters we found in the hyperparameter search.

3 Results

3.1 Architecture

When training the fully connected network, we chose for the hidden layer to have 128 nodes. The results for the fully connected layer are viewable in figure 3. The figure shows that the model quite quickly started overfitting, as the training loss kept decreasing but the test loss started increasing. The accuracy stayed at a relatively constant 23 percent.

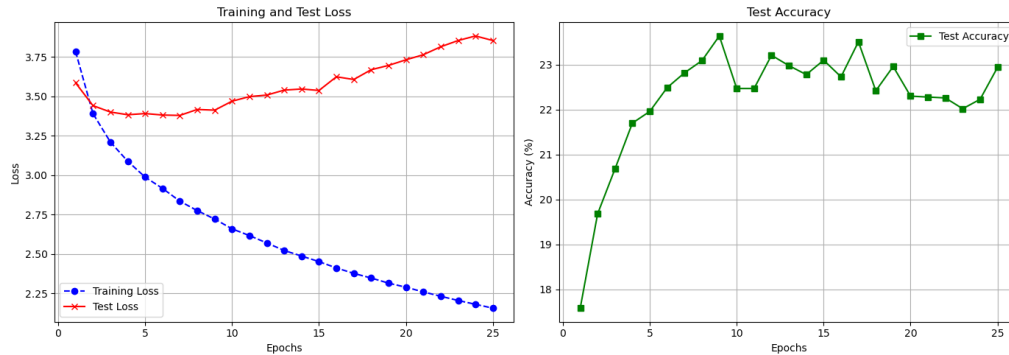


Figure 3: Training results of the two layer model with a hidden layer size of 128.

The convolutional model results can be seen in figure 4. These results show that the model took longer to overfit and ended with a higher accuracy of about 31 percent.

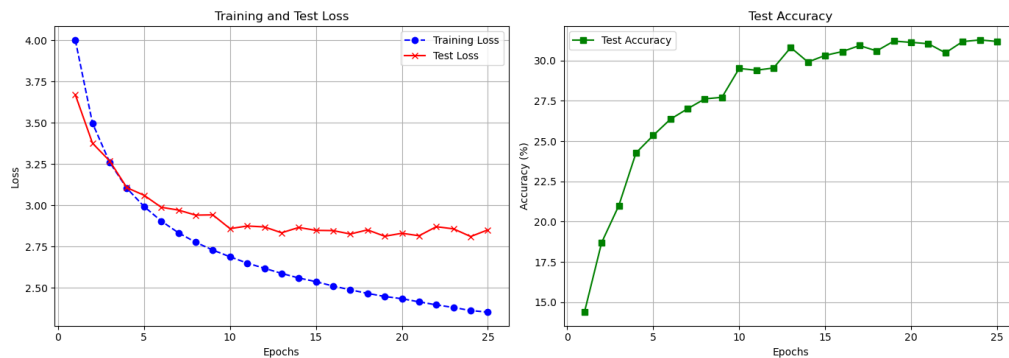


Figure 4: Training results for the base convolutional model.

The expanded models were trained after the hyperparameter search yielded the best parameters to do so with.

3.2 Hyperparameter search

There are too many images to display every result, so we will only show graphs for hyperparameters which yielded interesting results.

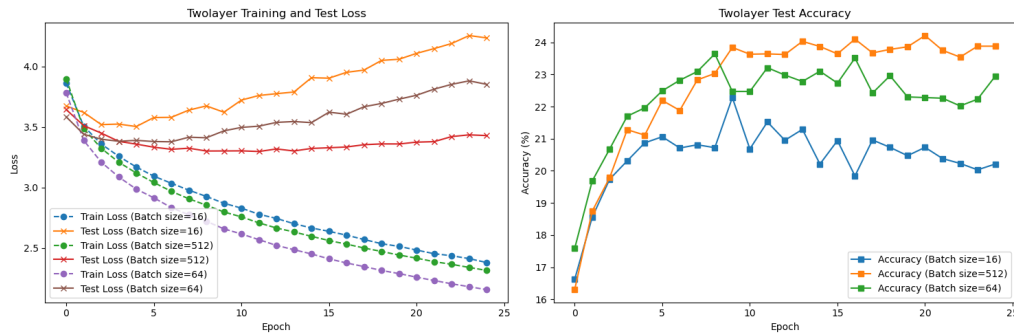


Figure 5: Batch size hyperparameter search for the two layer fully connected network.

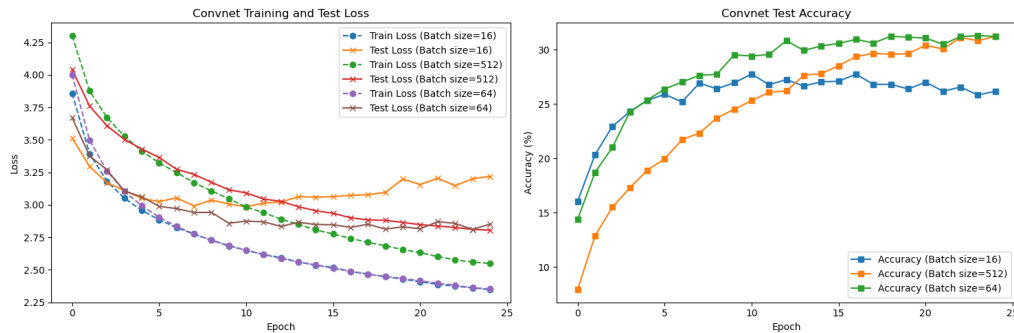


Figure 6: Batch size hyperparameter search for the two layer network.

- Optimizer:

As expected, the ADAM optimiser outperformed the SGD optimiser by a large margin. ADAM achieved a 10 percent higher accuracy for the fully connected network and a 30 percent higher accuracy for the convolutional network. This means that SGD did not manage to improve the convolutional network at all. We chose to keep using the ADAM optimiser.

- Learning rate:

The learning rate of 0.01 was too high, as both networks peaked at about 7.5 percent accuracy with this learning rate. The learning rate of 0.0001 was too low for the convolutional network to achieve similar accuracy to 0.01 in the 25 epochs it ran. The fully connected network however did achieve the same accuracy with 0.0001 as with 0.01. This is due to the fact that with a learning rate of 0.001 the fully connected network quickly overfits, while with a learning rate of 0.0001 the model only overfits later in the training. We chose to keep the learning rate at 0.001

- Batch size:

The results of the batch size training can be seen in figure 5 and 6. The batch size of 16 resulted in the lowest accuracy, with a high amount of overfitting. This is not what we expected, but may be due to the increased amount of weight updates leading the model to start overfitting earlier and at an increased rate. For both models batch size 512 showed the most stable increase to accuracy over time, even outperforming a batch size of 64 for the fully connected two layer model. We chose to keep batch size 512.

- Weight decay:

Both values we chose to test (0.01 and 0.1) were way too high for these models. Both resulted in a large decrease in accuracy on both the training and the test set. However, because weight decay is important for preventing overfitting, which was happening a lot, we decided to use weight decay 0.001 even though we did not test this during the search.

- Batch normalisation:

Batch normalisation increased the testset accuracy of the fully connected model from about 23 percent to about 25 percent. For the convolutional network it resulted in a faster convergence, but because of this also in a larger amount of overfitting. The results of the convolutional training can be seen in figure 7. Because of this we decided to leave batch normalisation off for the convolutional network, but keep it on for the fully connected network.

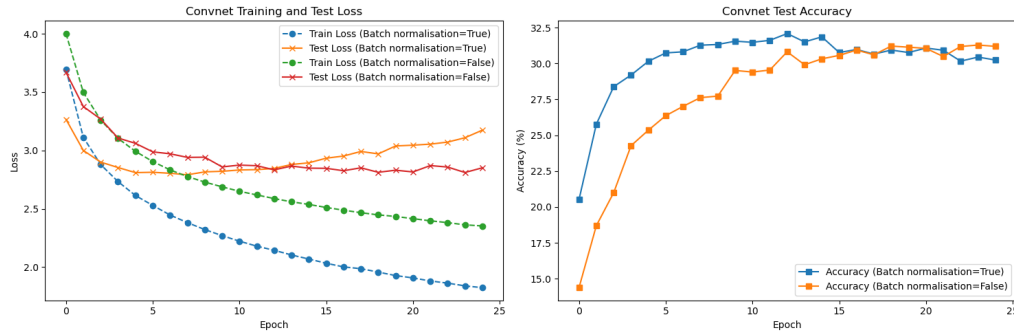


Figure 7: Results when training with batch normalisation on the convolutional network.

- Transformations:

The transformation preset with the most significant impact on model performance was idx 2. This is the preset with all randomisation, a shift of up to 4 pixels and a rotation of up to 30 degrees. It seems this images going through this transformation change too significantly when compared to the test set, as the convolutional network lost about 5 percent accuracy and the fully connected layer lost 12 percent accuracy. The transformation with idx 3 (all randomisation, 1 pixel shift, 5 degree rotation) resulted in the least overfit models, with a slightly better accuracy than no transformations. This can be seen in figure 8. We chose to use the transformation with idx 3.

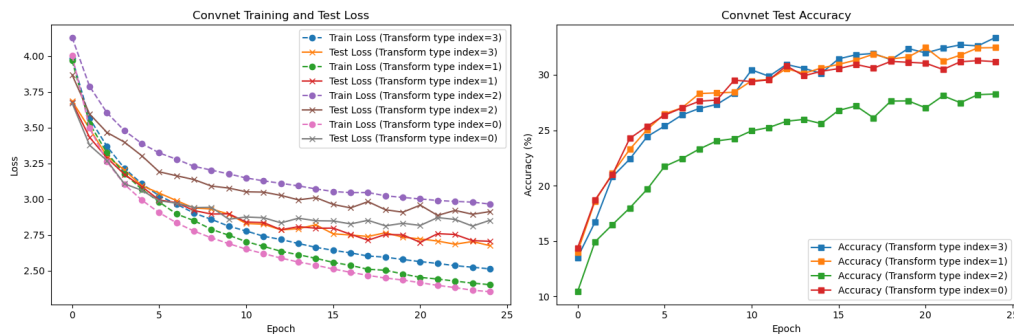


Figure 8: Training data of the convolutional network with the corresponding transformation set id as explained by figure 2.

3.3 Expanded models

For the expanded twolayer model, we decided to lower the amount of nodes in the hidden layers from 128 to 80, as the model was overfitting very quickly during the hyperparameter search. The training results of the expanded models are shown in figures 9 and 10. As is shown in the figures, this training ran for 100 epochs. The best accuracy for the convolutional model was 35.95 percent, while the fully connected model reached 33.95 percent.

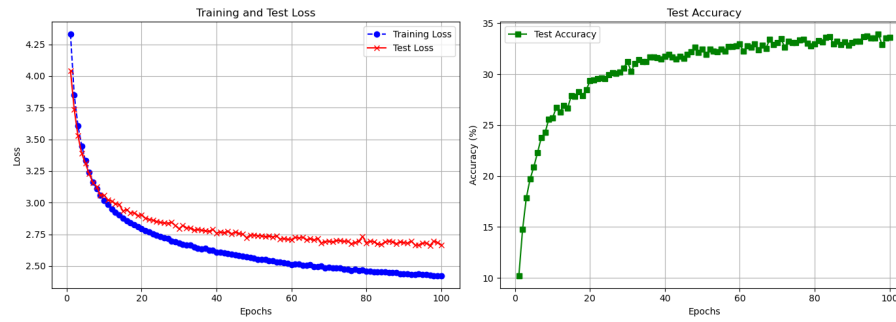


Figure 9: Training results of the expanded fully connected network (with a new convolutional layer) with all hyperparameters we chose before

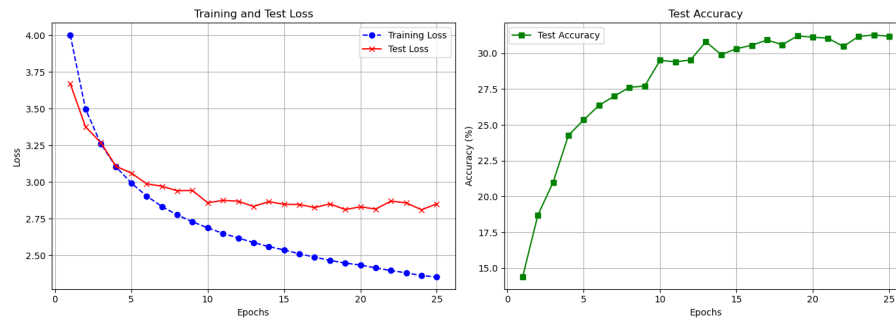


Figure 10: Training results of the expanded convolutional network with all hyperparameters we chose before

3.4 STL-10

To be able to use the convolutional model we trained before, we altered the final layer to output 5 classes. The input also gets put through an adaptive average pooling layer to always end up as size 32x32. As we are using an already trained model, we use the learning rate 0.00025, to increase stability. We also increase the shift in the transformation to 4 pixels and the rotation to 15 degrees. The weight decay was also increased to 0.01, because early testing showed exceedingly fast overfitting. The results from this training are shown in figure 11. The results show that accuracy increased fast until stagnating at about 56 percent due to overfitting. Our top accuracy was 56.65 percent. After testing some more values for the transformation and weight decay manually, this was the best model we could obtain. Training a model from scratch with the same architecture and parameters resulted in a worse model, as it was 10 percent less accurate after 50 epochs.

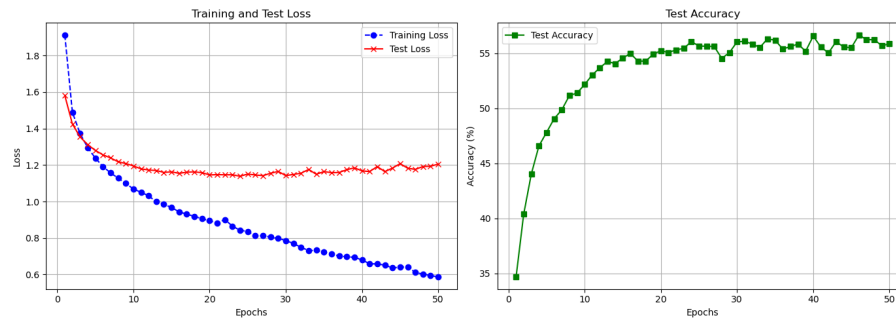


Figure 11: Performance of the transferred model on the STL 10 dataset.

A confusion matrix for the trained model is shown in figure 12. This shows that particu-

larly dogs and monkeys were harder for the model to identify. A 2 dimensional visualisation of the feature space generated using T-SNE is shown in figure 13. A larger concentration of horses is visible on the right side, while birds are split among the top left and bottom middle. A cluster of deer is also visible in the middle. The other areas of this feature space seem fairly evenly distributed, which could explain part of why dogs and monkeys were harder to identify for the model.

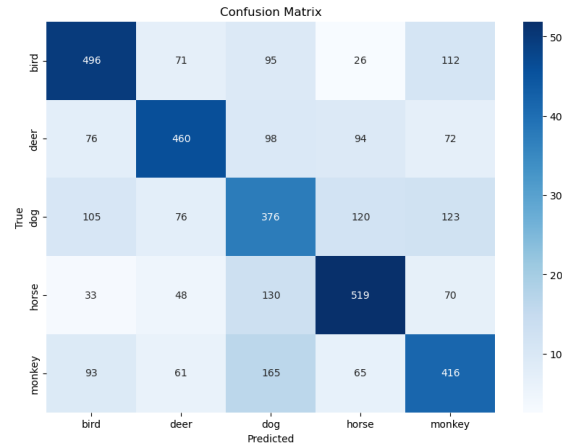


Figure 12: Confusion matrix for the convolutional model on the 5 classes of STL-10 that we trained on.

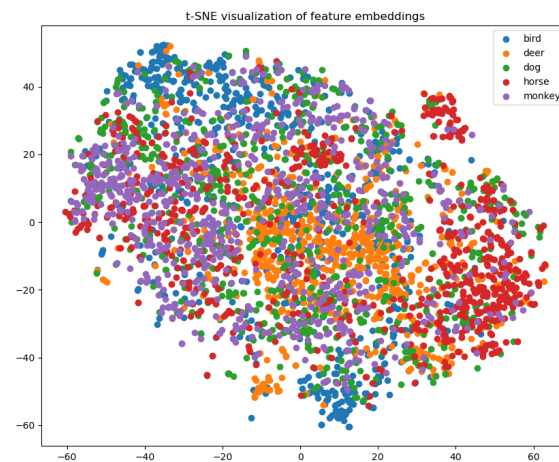


Figure 13: T-SNE visualisation of the feature space of the final layer of the convolutional model.

4 Discussion

In this report we showcased common techniques in image classification, such as convolutional neural networks, hyperparameter search and transfer learning. We achieved a top accuracy of 35.95 percent on the CIFAR-100 dataset and a top accuracy of 56.65 percent on half of the STL-10 dataset. Something that surprised us is that transfer learning turned out to work well even when the models involved were relatively small. For future models perhaps the accuracy could be increased if we added more layers along with skip connections.

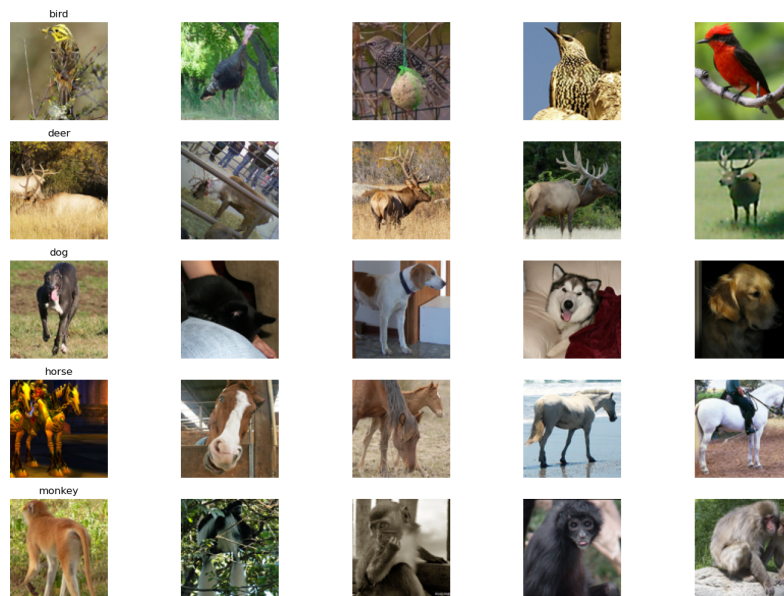


Figure 14: Visualisation showing 5 images of each class of STL-10 that we are training on.

A Large Images

References

- [1] Adam Coates, Andrew Ng, and Honglak Lee. “An analysis of single-layer networks in unsupervised feature learning”. In: *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2011, pp. 215–223.
- [2] Alex Krizhevsky, Geoffrey Hinton, et al. “Learning multiple layers of features from tiny images”. In: (2009).
- [3] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

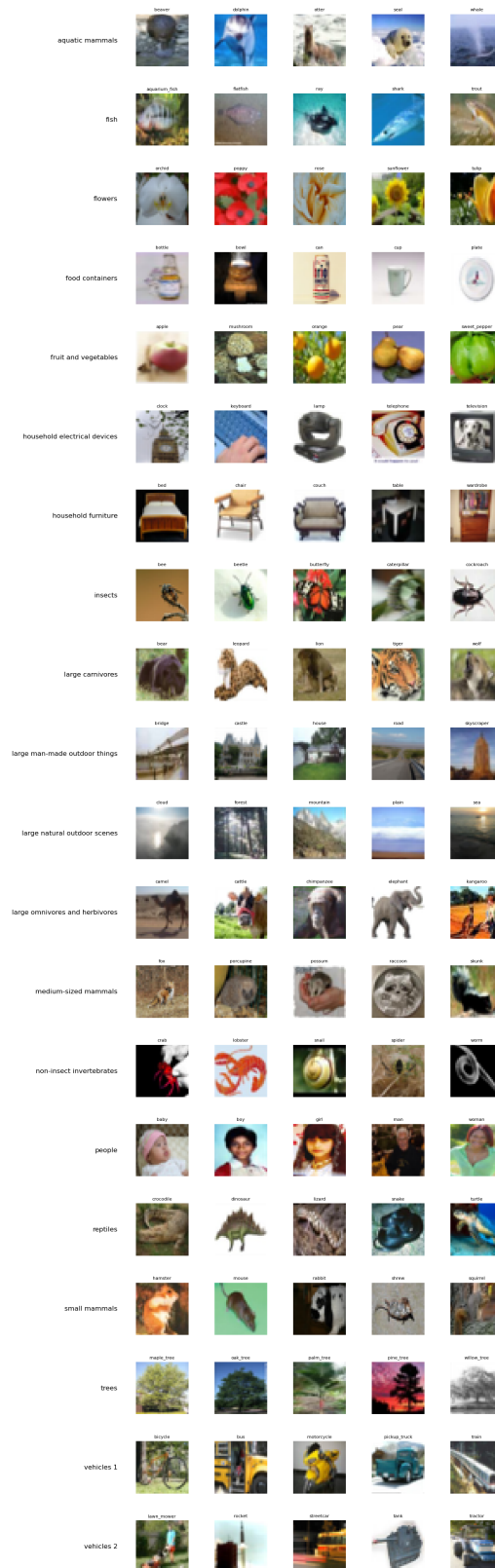


Figure 15: Visualisation showing an example image of every subclass of every superclass in the CIFAR-100 dataset.
